Resumen de la unidad

Resumen de la **Unidad 1 (semanas 1 a 4)**: el mapa de la unidad, cómo evoluciona la lectura de un botón y lo esencial de cada tema. Entre corchetes, la guía donde se explica.

Hardware · Software · código · Regla · decisión · Error · ojo · ◇ decisión

1 Mapa de la unidad

cada paso = una guía

1	**S1·1 Arquitectura [01]**	CPU, buses, memorias
2	**S1·2 Bits y registros [02]**	máscaras, DDR / PORT / PIN
3	**S2·1 ESP32 [03]**	GPIO, W1TS / W1TC, pines, superlazo
4	**S2·2 Programar para el hardware [04]**	tipos, `millis()`, pulsador
5	**S3 Temporizadores [05]**	prescaler, alarma, ISR
6	**S4 Interrupciones externas [06]**	`attachInterrupt`, IRAM, `volatile`, bandera

2 Leer un botón, de la S1 a la S4

cada semana lo hace de otra forma

1	**S1 · registro AVR**	leer el pin directo	`PIND & (1 << 2)`
2	**S2 · sondeo**	`digitalRead` + `millis()`: antirrebote y flanco	
3	**S3 · el hardware avisa solo**	ISR + `volatile` + bandera	
4	**S4 · `attachInterrupt`**	FALLING y antirrebote dentro de la ISR	

BITS Y REGISTROS AVR [02]

ENCENDER BIT N
```
REG |= (1 << n);
```

APAGAR BIT N
```
REG &= ~(1 << n);
```

INVERTIR BIT N
```
REG ^= (1 << n);
```

Pesos: 128 64 32 16 8 4 2 1 · 1 dígito hex = 4 bits · `0x20` = bit 5.

PINES AVR: DDR + PORT [02]

DDR 0 · PORT 0
Entrada flotante

DDR 0 · PORT 1
Entrada con pull-up: suelto 1, pulsado 0

DDR 1 · PORT 0
Salida 0 V

DDR 1 · PORT 1
Salida 5 V

ESP32 Y MEMORIAS [01][03]

ESP32 [03]

- DDR → `GPIO_ENABLE_REG`
- PORT → `W1TS` / `W1TC` (atómico)
- PIN → `GPIO_IN_REG`
- **3,3 V**, no tolera 5 V
- ≤ 10 mA por pin
- `R = (3,3 − Vf) / I`

PINES [03]

- **Seguros:** 16–19, 21–23, 25–27, 32, 33
- **Strapping:** 0, 2, 5, 12, 15
- **Solo entrada:** 34–36, 39
- **Flash:** 6–11

MEMORIAS [01]

Flash = programa · **SRAM** = variables (se borra) · **EEPROM** = datos que sobreviven.

TIPOS Y TIEMPO [04]

1. **Tipos de tamaño fijo** — `int` cambia entre placas: usar `uint8_t`, `uint32_t`.
2. **Tiempo sin bloquear** — `if (ahora - tAnterior >= PERIODO)` con `uint32_t`.
3. **Pulsador** — `INPUT_PULLUP` + antirrebote 20–50 ms + **flanco**.
4. **Desbordamiento** — Sin signo da la vuelta a 0; con signo es indefinido.

TEMPORIZADORES [05]

FRECUENCIA DE TICK	TIEMPO HASTA EL AVISO	ALARMA PARA UN TIEMPO
`f = 80 000 000 / N`	`tiempo = alarma / f`	`alarma = tiempo × f`

N = 80 → 1 tick = 1 µs → la alarma se escribe en µs. ESP32: 4 timers de 64 bits · AVR: 3 de 8/16 bits.

INTERRUPCIONES [05][06]

1. **Evento** — Timer (interno, predecible) o pin (externo, sin aviso).
2. **ISR corta con `IRAM_ATTR`** — Sin `Serial`, `delay` ni cálculos largos.
3. **Bandera `volatile`** — `bandera = true`
4. **`loop()`** — Baja la bandera y hace el trabajo.

PIN EXTERNO

`attachInterrupt(digitalPinToInterrupt(pin), isr, FALLING);` en `setup()`, **después** de `pinMode`. Pull-up + «al presionar» = **FALLING**.

ERRORES COMUNES

ERROR	CORRECTO
`REG = (1 << 2);` borra los demás bits	`REG |= (1 << 2);`
`int t = millis();`	`uint32_t t = millis();`
`millis() > t + P`	`millis() - t >= P`
`delay(200)` como antirrebote	Antirrebote con `millis()`
LED en GPIO34 (solo entrada)	Pin seguro: 16–19…
`Serial.println` en la ISR	Patrón bandera
Variable compartida sin `volatile`	`volatile`
ISR sin `IRAM_ATTR`	`void IRAM_ATTR isr()`